

Data Structures Lab – Pseudocode for All Experiments (SPPU)

Experiment 1A – Linear Search

```
BEGIN
  INPUT n
  DECLARE arr[n]

  FOR i ← 0 TO n-1
    INPUT arr[i]
  END FOR

  INPUT key
  flag ← 0

  FOR i ← 0 TO n-1
    IF arr[i] = key THEN
      PRINT "Student Found"
      flag ← 1
      BREAK
    END IF
  END FOR

  IF flag = 0 THEN
    PRINT "Student Not Found"
  END IF
END
```

Experiment 1B – Binary Search

```
BEGIN
  INPUT n
  DECLARE arr[n]

  FOR i ← 0 TO n-1
    INPUT arr[i]
  END FOR

  INPUT key

  low ← 0
  high ← n-1
  flag ← 0

  WHILE low ≤ high
    mid ← (low + high)/2

    IF arr[mid] = key THEN
      PRINT "Student Found"
      flag ← 1
      BREAK

    ELSE IF key > arr[mid] THEN
```

```

        low ← mid + 1

    ELSE
        high ← mid - 1
    END IF
END WHILE

IF flag = 0 THEN
    PRINT "Student Not Found"
END IF
END

```

Experiment 2A – Bubble Sort

```

BEGIN
    INPUT n
    DECLARE a[n]

    FOR i ← 0 TO n-1
        INPUT a[i]
    END FOR

    FOR i ← 0 TO n-2
        FOR j ← 0 TO n-i-2

            IF a[j] > a[j+1] THEN
                temp ← a[j]
                a[j] ← a[j+1]
                a[j+1] ← temp
            END IF

        END FOR
    END FOR

    PRINT "Sorted Array"

    FOR i ← 0 TO n-1
        PRINT a[i]
    END FOR
END

```

Experiment 2B – Selection Sort

```

BEGIN
    INPUT n
    DECLARE a[n]

    FOR i ← 0 TO n-1
        INPUT a[i]
    END FOR

    FOR i ← 0 TO n-2

        min ← i

        FOR j ← i+1 TO n-1

```

```

        IF a[j] < a[min] THEN
            min ← j
        END IF
    END FOR

    temp ← a[i]
    a[i] ← a[min]
    a[min] ← temp

END FOR

PRINT "Sorted Array"

FOR i ← 0 TO n-1
    PRINT a[i]
END FOR
END

```

Experiment 3 – Hash Table

```

BEGIN
    CREATE hash table

    INPUT name, phone

    index ← HASH(name)

    IF slot is empty THEN
        INSERT record

    ELSE

        APPLY collision resolution

        Linear Probing:
            newIndex ← (index + i) MOD table_size

        Quadratic Probing:
            newIndex ← (index + i2) MOD table_size

        Double Hashing:
            newIndex ← (index1 + i × index2) MOD table_size

        REPEAT until empty slot found

        INSERT record

    END IF

    SEARCH record

    IF found THEN
        PRINT phone number
    ELSE
        PRINT "Record Not Found"
    END IF
END

```

Experiment 4 – Pinnacle Club using Linked List

```
BEGIN
    CREATE node with:
        PRN
        Name
        Next Pointer

    head ← NULL

    ADD President at beginning

    ADD Secretary at end

    ADD members between President and Secretary

    DELETE President from beginning

    DELETE Secretary from end

    DELETE member using PRN

    count ← 0

    temp ← head

    WHILE temp ≠ NULL
        count ← count + 1
        temp ← temp.next
    END WHILE

    DISPLAY all members
END
```

Experiment 5 – Theatre Ticket Booking

```
BEGIN
    CREATE doubly linked list

    INITIALIZE all seats as available

    DISPLAY available seats

    INPUT seat number to book

    IF seat available THEN
        REMOVE seat from available list
        ADD seat to booked list
        PRINT "Seat Booked"
    ELSE
        PRINT "Seat Not Available"
    END IF

    INPUT seat number to cancel

    IF seat booked THEN
        REMOVE seat from booked list
        ADD seat to available list
    END IF
END
```

```
        PRINT "Booking Cancelled"

    ELSE
        PRINT "Seat Not Booked"
    END IF
END
```

Experiment 6A – Infix to Postfix Conversion

```
BEGIN
    INPUT infix expression

    INITIALIZE empty stack

    FOR each symbol in expression

        IF operand THEN
            ADD to postfix

        ELSE IF '(' THEN
            PUSH into stack

        ELSE IF operator THEN

            WHILE precedence(top) ≥ precedence(current)
                POP from stack
                ADD to postfix
            END WHILE

            PUSH operator

        ELSE IF ')' THEN

            WHILE top ≠ '('
                POP and add to postfix
            END WHILE

            POP '('

        END IF

    END FOR

    POP remaining operators

    DISPLAY postfix expression
END
```

Experiment 6B – Postfix Evaluation

```
BEGIN
    INPUT postfix expression

    INITIALIZE empty stack

    FOR each symbol in postfix
```

```

        IF operand THEN
            PUSH into stack

        ELSE

            operand2 ← POP
            operand1 ← POP

            result ← operand1 operator operand2

            PUSH result

        END IF

    END FOR

    PRINT final result
END

```

Experiment 7 – Pizza Parlor Queue

```

BEGIN
    INITIALIZE queue

    INPUT maximum size M

    REPEAT

        DISPLAY menu

        INPUT choice

        CASE choice

            1:
                IF queue full THEN
                    PRINT "Queue Full"
                ELSE
                    INSERT order at rear
                END IF

            2:
                IF queue empty THEN
                    PRINT "Queue Empty"
                ELSE
                    DELETE order from front
                END IF

            3:
                DISPLAY all pending orders

            4:
                EXIT

        END CASE

    UNTIL choice = 4
END

```

Experiment 8 – Binary Search Tree Operations

```
BEGIN
    CREATE root node

    INSERT nodes according to BST rules

    FOR longest path
        CALCULATE height recursively

    FOR minimum value
        TRAVERSE leftmost node

    FOR mirror tree
        SWAP left and right child recursively

    FOR searching
        IF key < node
            MOVE left
        ELSE
            MOVE right
        END IF

    DISPLAY inorder traversal
END
```

Experiment 9 – Graph using Adjacency List

```
BEGIN
    CREATE adjacency list

    INPUT number of flights

    FOR each flight
        INPUT source city
        INPUT destination city
        INPUT cost

        ADD edge between cities
    END FOR

    DISPLAY adjacency list

    MARK all vertices unvisited

    PERFORM DFS traversal

    IF all vertices visited THEN
        PRINT "Graph Connected"
    ELSE
        PRINT "Graph Not Connected"
    END IF
END
```
